

RideFlow Ubuntu Developer and Operations Guide

Repository: <https://github.com/raiven427/ride-flow>

Scope: fresh-installation UI state, admin-only operations monitoring, online presence, activity feed, self-hosting, and safe maintenance.

This guide explains the new changes and how to operate the code on Ubuntu. It does not claim that payment, transport, legal, or safety compliance is complete merely because the application runs.

1. What changed

RideFlow now opens with a clean new-installation presentation. Dashboard metrics show zero savings, zero favorite rides, zero safety check-ins, and a zero fare placeholder until a real quote is created. The existing admin fare controls, notification settings, ownership transfer, payment readiness, and deployment configuration were preserved.

An admin-only **Admin operations** view was added to the sidebar. It shows the total number of authenticated accounts, users with a recent heartbeat, and recent activity events. The people table shows each account's name, email, role, presence state, and current view. The activity feed shows recent presence and application events.

Presence is intentionally approximate. A signed-in browser sends a heartbeat every 30 seconds. The server considers a user online when the last heartbeat is within 90 seconds. A browser that is closed, loses connectivity, or goes idle may remain online until that window expires. This is not a substitute for a dedicated realtime presence service.

2. Where the new code lives

File	What it does	How to change it
<code>client/src/pages/Home.tsx</code>	Displays zero-state overview metrics, sends browser heartbeats, adds the admin navigation item, and renders the operations view	Edit UI labels, polling interval, view names, and dashboard layout
<code>client/src/index.css</code>	Styles operations cards, presence dots, activity rows, and responsive behavior	Edit colors, spacing, breakpoints, and visual states
<code>server/routers.ts</code>	Defines <code>presence.heartbeat</code> , <code>presence.activity</code> , and <code>admin.operations</code> tRPC procedures	Add or protect new admin actions here
<code>server/db.ts</code>	Writes presence/activity records and joins users to presence for the admin snapshot	Change query filters, freshness rules, and event retention logic
<code>drizzle/schema.ts</code>	Defines <code>rideflow_presence</code> and <code>rideflow_activity_events</code>	Add columns only through reviewed migrations
<code>drizzle/0004_calm_leader.sql</code>	Creates the two new tables	Apply to a new host database;

File	What it does	How to change it
		never edit an applied migration casually
<code>server/operations.test.ts</code>	Proves admins can access operations and ordinary users cannot	Add authorization and data-shape regression tests

3. Database tables

`rideflow_presence` stores one current row per user. Its unique `userId` prevents duplicate presence rows. `status` is `online`, `away`, or `offline`; `currentView` records the current workspace label; and `lastSeenAt` is the freshness timestamp.

`rideflow_activity_events` is an append-only event stream. It stores the event type, a short human-readable summary, optional JSON metadata, the related user, and creation time. Do not store passwords, payment secrets, identity-document contents, or sensitive payment payloads in event metadata.

Before a production launch, add a retention policy. For example, keep detailed activity for a defined period and retain only aggregated counts afterward. If you add scheduled cleanup, use your own host's job system and document the retention decision.

4. Admin workflow

Sign in with the verified admin account. The sidebar displays **Admin operations** only when the server session has `role = admin`. The frontend visibility is only a convenience; the server's `adminProcedure` is the real security boundary.

Open **Admin operations** to inspect the account count, online count, and recent activity. Refreshing happens automatically every 15 seconds. If no users have signed in since the migration, the page shows an intentional empty state rather than invented people or activity.

Open **Profile** to use the existing Admin Control Room. Fare rules, commission, notification email, and ownership transfer remain there. Changing a fare requires the existing admin

procedure and affects future server-side quotes; it does not rewrite historical quotes or ledger entries.

5. Ubuntu setup

On Ubuntu 24.04, install the basic toolchain:

```
sudo apt update
sudo apt install -y git curl build-essential mysql-client
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
corepack enable
corepack prepare pnpm@10.4.1 --activate
node --version
pnpm --version
```

Clone the private repository and install dependencies:

```
git clone https://github.com/raiven427/ride-flow.git
cd ride-flow
pnpm install
```

Create a local secret file from `.env.example` only on the server or local machine. Do not commit it:

```
cp .env.example .env
chmod 600 .env
nano .env
```

Configure your own database, `JWT_SECRET`, OIDC provider, private S3-compatible storage, notification webhook, and sandbox payment values. The exact variables are documented in `docs/deployment-environment.template` and `docs/rideflow-deployment-payments-guide.md`.

6. Database setup and migration

Provision a MySQL or TiDB-compatible database, place its connection string in `DATABASE_URL`, and run the reviewed migrations. For a fresh database, inspect generated SQL before applying it:

```
pnpm drizzle-kit generate
# inspect drizzle/*.sql
pnpm drizzle-kit migrate
```

For a database that already contains RideFlow data, back it up first. Never use `DROP DATABASE`, `TRUNCATE`, or an unreviewed destructive migration. Test a restore on a separate database before changing production.

The new migration is additive. It creates `rideflow_presence` and `rideflow_activity_events` and does not delete existing users, fare rules, quotes, files, admin settings, or ledger rows.

7. Run and verify the application

Use the development server:

```
pnpm dev
```

In another terminal, check the service:

```
curl -i http://localhost:3000/healthz
pnpm check
pnpm test
pnpm build
```

A successful local test suite should include the operations authorization tests. Test as an ordinary user to confirm the Admin operations navigation is absent and the server rejects direct `admin.operations` calls. Test as the admin to confirm the page loads and refreshes.

8. How to update the app safely

Create a branch before editing:

```
git checkout main
git pull --ff-only origin main
git checkout -b feature/your-change
```

Make a focused change. If you change a database table, update `drizzle/schema.ts`, generate and review SQL, apply it first to a disposable database, and add a test. If you change a tRPC procedure, update the UI and add authorization coverage. If you change payment code, use sandbox credentials and test provider webhooks before production.

Run the checks and commit:

```
pnpm check
pnpm test
pnpm build
git add -A
git commit -m "Describe the change"
git push -u origin feature/your-change
```

Merge only after reviewing the diff and the test result. Keep `main` deployable. Never commit `.env`, API keys, database dumps, private identity documents, or webhook signing secrets.

9. Common changes

To change the visible fresh-state metrics, edit the metric values in the `Overview` component in `client/src/pages/Home.tsx`. Keep them at zero for a new installation; production analytics should come from real database queries rather than fabricated numbers.

To change the online freshness window, edit the 30-second client heartbeat interval and the 90-second server cutoff together. The server cutoff is in `getAdminOperationsSnapshot` in `server/db.ts`. Make the values explicit in a comment and add a test if the policy changes.

To add an activity event, call the protected `presence.activity` mutation from a user action or call `recordActivity` from a server-side event. Keep summaries short and avoid sensitive data.

To add another admin metric, extend the return value of `getAdminOperationsSnapshot`, expose only aggregated or necessary fields, and render the metric in `OperationsDashboard`. Add a test proving ordinary users cannot access it.

10. Production hardening still required

The dashboard is useful operational visibility, but it is not yet a full Uber-style control center. Before real operation, add a dedicated realtime channel or managed presence service, rate-limit heartbeats, add server-side audit logging for administrator actions, paginate activity, and implement retention. Add dispatch, live GPS, maps, driver availability, trip state transitions, chat, scheduled rides, refunds, payout reconciliation, fraud detection, support tooling, and incident response.

Do not use the dashboard as the sole source of truth for money or safety decisions. Financial state belongs in the ledger and provider reconciliation. Safety decisions require verified identities, driver checks, emergency workflows, and trained operations staff.

11. References

[1]: <https://ubuntu.com/server/docs> Ubuntu Server documentation [2]:
<https://orm.drizzle.team/docs/migrations> Drizzle migration documentation [3]:
https://openid.net/specs/openid-connect-core-1_0.html OpenID Connect Core specification [4]:
<https://docs.stripe.com/webhooks> Stripe webhook documentation [5]:
<https://developer.safaricom.co.ke/> Safaricom Daraja developer portal